

# Verifying Distributed Algorithms with Interactive Theorem Provers: A Survey of Approaches

MAXWELL MOIR, University of Canterbury, New Zealand

Proving the correctness of programs is not only elegant in a theoretical sense, but is often necessary in mission-critical applications. However, in specific circumstances where correctness must be ensured, formal proofs may be required. This survey reviews the different approaches to verifying the correctness of distributed algorithms.

CCS Concepts: • **Theory of computation** → **Logic and verification**.

Additional Key Words and Phrases: distributed systems, formal verification, proof assistants

## ACM Reference Format:

Maxwell Moir. 2026. Verifying Distributed Algorithms with Interactive Theorem Provers: A Survey of Approaches. *J. ACM* 000, 0, Article 000 (2026), 4 pages. <https://doi.org/XXXXXXX.XXXXXXX>

## 1 INTRODUCTION

With an increasing societal dependance on software, ensuring the effective operation of the tools we use has become more important. One method for reasoning about the correctness of programs is formal verification. Distributed systems underpin many of the applications that we use every day. Distributed systems are notoriously difficult to verify due to ...

This survey aims to explore attempts to formally verify distributed algorithms, both theoretically and in practice. It provides a taxonomy of different theorem-based verification approaches and frameworks, reviews several common tools, and contrasts with other verification approaches.

Other approaches to the verification of distributed algorithms are outside the scope of this survey, except when used as comparison.

## 2 PRELIMINARY

### 2.1 Distributed Protocols

Distributed computing is a fundamental field in Computer Science. It concerns the inter-communication between distinct autonomous components in a system. This patterns shows up often, usually when there is a network of multiple devices.

A common challenge in distributed computing is fault tolerance. This underpins some of the problems that are often the targets of formal verification. For example, consensus protocols such as Paxos and Raft. Another example is Byzantine Fault Tolerance.

---

Author's address: Maxwell Moir, University of Canterbury, Christchurch, New Zealand, mmo199@uclive.ac.nz.

---

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 0004-5411/2026/0-ART000

<https://doi.org/XXXXXXX.XXXXXXX>

## 2.2 Interactive Theorem Provers

Interactive Theorem Provers, or proof assistants, have seen a recent surge in popularity among computer scientists and mathematicians alike. Proof assistants are tools used to assist with the human development of formal proofs. These formal proofs can then be automatically checked by the machine, ensuring correctness at each stage of the process. Theorem provers are extremely useful in the formal verification of protocols, where they can be used to ensure specific properties of a system hold. For this reason proof assistants such as Coq, Lean, Isabelle, and Dafny have become popular for verifying distributed algorithms.

## 2.3 Verification

Verifying the correctness of a program requires that specific conditions about a system can be shown to hold in certain states. These conditions can be arbitrarily chosen, but usually represent ... The two most notable property categories of a system are “Safety” and “Liveness”. A safety property ensures protection against unfavourable events, whereas liveness properties make sure that the program functions as intended.

Fault tolerance is another notable aspect of ensuring a program works as intended. Byzantine fault tolerance, specifically, is a common problem in

## 3 APPROACHES

Byzantine fault tolerance

Compositional and modular verification

Proof techniques: Invariant refinement

### 3.1 Tools

### 3.2 Common Frameworks

There are several common frameworks used in the verification of distributed protocols. Each of which has its own benefits and detractions.

Verdi [7] Iron Fleet [3] Diesel [6] Veil [4] TLAPS [2]

### 3.3 Proof techniques

### 3.4 Proof techniques

## 4 APPLICATIONS

Nam id fermentum dui. Suspendisse sagittis tortor a nulla , in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis

### 4.1 Verified protocols

Nam id fermentum dui. Suspendisse sagittis tortor a nulla , in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

BFT [8]

### 4.2 Verified software

AWS [1] Mongo [5]

Nam id fermentum dui. Suspendisse sagittis tortor a nulla , in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut suscipit

## 5 ALTERNATIVE VERIFICATION APPROACHES

Model checking, congue. Quisque mattis elit a risus commodo venenatis eget dui. Etiam sagittis eleifend elementum.

## 6 FUTURE DIRECTIONS

One of the common criticisms of formal verification is that the effort of verifying a system is unpractical and unnecessary. ... AI

## 7 CONCLUSION

Nam id fermentum dui. sagittis tortor a nulla mollis, in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum. Test 01

## 8 METHODOLOGY

### 8.1 Literature Search Strategy

To identify relevant papers, we used a systematic literature search strategy. We chose to query two databases, the ACM digital library and IEEE Xplore. These databases were accessed on April 17 2026.

To ensure the quality of of this review, we decided to follow the PRISMA framework to inform our search strategy. The search strategy can be summarized into multiple steps. First, we curated relevant keywords and constructed seach queries to compile relevant papers based on the guiding questions of the survey. This led to the following seach query:

("proof assistant" OR "theorem prover" OR "mechanized verification" OR "formal verification") AND ("distributed protocol" OR "distributed algorithm")

This query yeilded 213 results from the ACM Digital Library and 49 results from IEEE Xplore.

### 8.2 Inclusion and Exclusion Criteria

These results were then screened based on standard inclusion and exclusion requirements. Papers primarily focusing on model checking and duplicates were removed. In addition, only papers that explicitly concerned distributed systems and proof assistants were included. This reduced the result count to 37 for ACM and XX for IEEE Xplore. From this point, papers commonly cited in these results were added. From these, the most relevant papers were selected from the remainders.

## REFERENCES

- [1] Marc Brooker and Ankush Desai. 2025. Systems Correctness Practices at Amazon Web Services. *Commun. ACM* 68, 6 (June 2025), 38–42. <https://doi.org/10.1145/3729175>
- [2] Denis Cousineau, Damien Doligez, Leslie Lamport, Stephan Merz, Daniel Ricketts, and Hernan Vanzetto. 2012. TLA+ Proofs. *Proceedings of the 18th International Symposium on Formal Methods (FM 2012)*, Dimitra Giannakopoulou and Dominique Mery, editors. Springer-Verlag Lecture Notes in Computer Science 7436 (Jan. 2012), 147–154.
- [3] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. 2015. IronFleet: Proving Practical Distributed Systems Correct. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15)*. Association for Computing Machinery, New York, NY, USA, 1–17. <https://doi.org/10.1145/2815400.2815428>
- [4] George Pirlea, Vladimir Gladshstein, Elad Kinsbruner, Qiyuan Zhao, and Ilya Sergey. 2025. Veil: A Framework for Automated and Interactive Verification of Transition Systems. In *Computer Aided Verification: 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part III*. Springer-Verlag, Berlin, Heidelberg, 26–41. [https://doi.org/10.1007/978-3-031-98682-6\\_2](https://doi.org/10.1007/978-3-031-98682-6_2)

- [5] William Schultz, Ian Dardik, and Stavros Tripakis. 2022. Formal Verification of a Distributed Dynamic Reconfiguration Protocol. In *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2022)*. Association for Computing Machinery, New York, NY, USA, 143–152. <https://doi.org/10.1145/3497775.3503688>
- [6] Ilya Sergey, James R. Wilcox, and Zachary Tatlock. 2017. Programming and Proving with Distributed Protocols. *Proc. ACM Program. Lang.* 2, POPL (Dec. 2017), 28:1–28:30. <https://doi.org/10.1145/3158116>
- [7] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. 2015. Verdi: A Framework for Implementing and Formally Verifying Distributed Systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*. Association for Computing Machinery, New York, NY, USA, 357–368. <https://doi.org/10.1145/2737924.2737958>
- [8] Qiyuan Zhao, George Pirlea, Karolina Grzeszkiewicz, Seth Gilbert, and Ilya Sergey. 2024. Compositional Verification of Composite Byzantine Protocols. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. Association for Computing Machinery, New York, NY, USA, 34–48. <https://doi.org/10.1145/3658644.3690355>